GIT, GESTION DE PROJET ET AGILITÉ · JOUR 4 SUR 5 · JEUDI 29 OCTOBRE

Jeudi · Gestion de projet et agilité

Depuis lundi, vous savez **comment** travailler à plusieurs sur du code. Reste à savoir **quoi** faire, dans quel ordre, et comment s'organiser pour livrer quelque chose qui sert. C'est le rôle de la gestion de projet.

Aujourd'hui sert aussi à préparer demain. En fin de journée, chaque équipe aura le plan de travail de son projet, prêt dans GitHub.

1. Un projet, c'est quoi ?

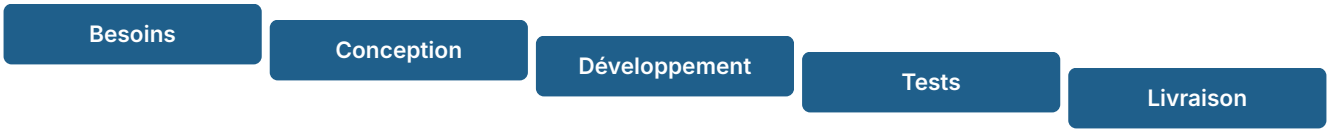
Un projet, c'est un objectif à atteindre, avec un début et une fin, et des moyens limités. On résume souvent ses contraintes par un triangle :

Contrainte	La question	Exemple pour une appli de covoiturage
Périmètre	Qu'est-ce qu'on livre ?	inscription, recherche de trajet, réservation
Délai	Pour quand ?	une première version en 3 mois
Coût	Avec quels moyens ?	4 développeurs, 120 000 €

Les trois sont liés. Si le client ajoute une fonctionnalité (périmètre), il faut plus de temps ou plus de monde. Si on réduit le délai sans toucher au reste, c'est la **qualité** qui trinque : bugs, code bâclé, équipe épuisée.

2. La méthode en cascade

C'est la façon « classique » de mener un projet : on enchaîne des étapes, chacune une seule fois, dans l'ordre.



Le cycle en V ajoute, en face de chaque étape de conception, l'étape de test qui la vérifiera.

Ce qui marche bien : quand le besoin est connu à l'avance et ne bougera pas. Construire un pont, un logiciel imposé par une réglementation, un contrat au périmètre figé. Le cadre est clair et on sait ce qu'on signe.

Le problème : le client ne voit le produit qu'à la fin. Si on a mal compris son besoin au départ, ou si ce besoin a changé en route, on le découvre au moment de la livraison. On appelle ça **l'effet tunnel**. Revenir en arrière coûte alors très cher.

3. L'agilité

En 2001, dix-sept développeurs se réunissent à Snowbird, dans l'Utah, et publient le **Manifeste agile**. Ils y défendent quatre valeurs. Pour chacune, ils ne rejettent pas le second élément : ils donnent la priorité au premier.

Plus important…	…que
les personnes et leurs échanges	les processus et les outils
un logiciel qui fonctionne	une documentation exhaustive
la collaboration avec le client	la négociation du contrat
s'adapter au changement	suivre un plan à la lettre

Le texte complet, avec ses douze principes, est sur agilemanifesto.org (traduction française disponible).

Concrètement, on ne livre plus tout à la fin. On découpe le projet en **itérations** courtes, de une à quatre semaines. À la fin de chacune, on livre une version qui fonctionne, même incomplète, et on la montre au client. Ses retours orientent l'itération suivante.

Une image connue résume l'idée. Si le client veut une voiture, l'approche en cascade livre une roue, puis un châssis, puis une carrosserie : rien d'utilisable avant la toute fin. L'approche agile livre d'abord une trottinette, puis un vélo, puis une moto, puis la voiture. À chaque étape, le client peut se déplacer, et dire ce qui lui manque.

4. Scrum

L'agilité est un état d'esprit. **Scrum** est la méthode agile la plus utilisée pour l'appliquer. Elle est décrite dans un court document gratuit, le *Scrum Guide* (version de 2020).

L'équipe

Une équipe Scrum compte en général 10 personnes au maximum, avec trois responsabilités :

Rôle	Ce qu'il fait
Product Owner	représente le client. Décide de ce qu'on construit et dans quel ordre, et gère la liste des choses à faire (le *backlog*).
Scrum Master	veille à ce que l'équipe applique Scrum, anime les réunions, et l'aide à lever ce qui la bloque. Ce n'est pas un chef.
Développeurs	construisent le produit. Ils décident eux-mêmes comment s'organiser pour atteindre l'objectif du sprint.

Les événements

Le travail est découpé en **sprints** : des périodes fixes d'un mois au maximum (souvent deux semaines), qui s'enchaînent sans pause.

Événement	Quand	À quoi ça sert
Sprint Planning	au début du sprint	choisir ce qu'on fait pendant le sprint et fixer un objectif
Daily Scrum	chaque jour, 15 minutes max	faire le point : où on en est, ce qui bloque
Sprint Review	à la fin du sprint	montrer ce qui a été fait aux utilisateurs et recueillir leurs retours
Sprint Retrospective	juste après la review	améliorer la façon de travailler de l'équipe

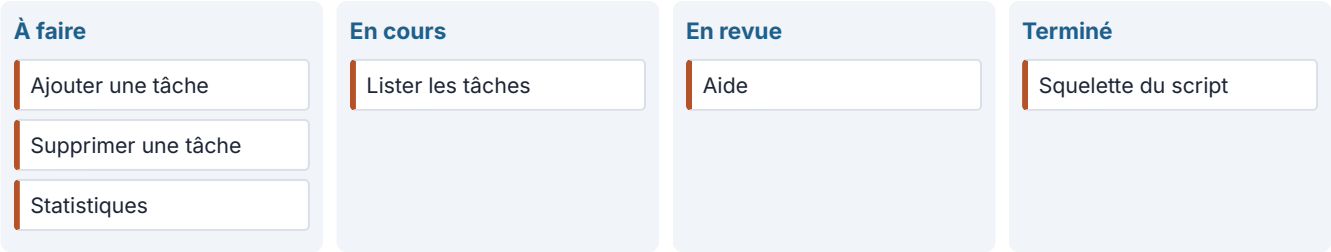
Les artefacts

Artefact	C'est quoi
Product Backlog	la liste ordonnée de tout ce qu'il faudrait faire sur le produit. Le Product Owner la tient à jour.
Sprint Backlog	les éléments choisis pour le sprint en cours, et le plan pour les réaliser
Incrément	ce qui a été terminé pendant le sprint, et qui fonctionne

Un élément n'est « terminé » que s'il respecte la **Definition of Done** (DoD), une liste de critères que l'équipe fixe ensemble. Par exemple : le code est sur `main`, il a été relu dans une PR, il a été testé, le README est à jour.

5. Kanban

Kanban est plus léger : pas de sprints, pas de rôles imposés. Un tableau, des colonnes, des cartes qui avancent de gauche à droite.



La règle qui change tout : **limiter le travail en cours** (le *WIP*, pour *work in progress*). Par exemple, pas plus de deux cartes par personne dans « En cours ». On termine avant de commencer autre chose. Sans cette limite, tout le monde commence plein de choses et rien n'est jamais fini.

Beaucoup d'équipes mélangent les deux : des sprints Scrum, et un tableau Kanban pour suivre le travail du sprint. C'est ce qu'on fera demain.

6. Les user stories

Une **user story** décrit un besoin du point de vue de l'utilisateur, en une phrase :

> En tant que **qui**, je veux **quoi**, afin de **pourquoi**.

Par exemple : *En tant que joueur, je veux voir les lettres déjà proposées, afin de ne pas proposer deux fois la même.*

Le « afin de » est la partie qu'on oublie le plus souvent, et la plus utile. C'est elle qui permet au Product Owner de juger si la story vaut la peine, et à l'équipe de trouver la bonne solution.

Une bonne story respecte les critères **INVEST** :

Lettre	Critère	Ce que ça veut dire
I	Indépendante	on peut la réaliser sans attendre une autre story
N	Négociable	elle décrit un besoin, pas une solution technique figée
V	de Valeur	elle apporte quelque chose à un utilisateur
E	Estimable	l'équipe peut dire à peu près combien de travail elle représente
S	*Small* (petite)	elle tient dans un sprint, idéalement en quelques jours ou moins
T	Testable	on peut vérifier qu'elle est terminée

Les critères d'acceptation

Pour rendre une story testable, on lui ajoute des **critères d'acceptation**, souvent sous la forme *Étant donné / Quand / Alors* :

> **Étant donné** que le mot à trouver est « boucle » et que j'ai déjà joué la lettre « e », **quand** je propose de nouveau la lettre « e », **alors** le jeu m'affiche « Déjà jouée » et ne compte pas d'erreur.

Une story est terminée quand tous ses critères sont vérifiés et que la Definition of Done est respectée.

7. Estimer et prioriser

Estimer en points

On estime rarement en heures : on se trompe presque toujours. On compare plutôt les stories entre elles, en **points**, avec une échelle inspirée de la suite de Fibonacci : 1, 2, 3, 5, 8, 13. Une story à 2 points représente à peu près deux fois le travail d'une story à 1 point. L'écart entre les valeurs grandit volontairement : plus une tâche est grosse, moins l'estimation est précise. Au-delà de 13, il faut découper la story.

Le **planning poker** se joue en équipe :

1. Le Product Owner lit une story, l'équipe pose ses questions.
2. Chacun choisit sa carte **en secret**, puis tout le monde la montre en même temps.
3. Si les estimations sont éloignées, la personne la plus basse et la plus haute expliquent leur choix. Souvent, l'une d'elles a vu une difficulté que les autres ont ratée.
4. On revote jusqu'à se mettre d'accord.

Après quelques sprints, on connaît sa **vélocité** : le nombre de points que l'équipe termine en moyenne par sprint. Elle sert à savoir combien de stories prendre au sprint suivant.

Prioriser avec MoSCoW

Catégorie	Sens	Règle
Must have	indispensable	sans elle, le produit ne sert à rien
Should have	important	on la veut, mais on peut livrer sans
Could have	bonus	si on a le temps
Won't have	pas cette fois	écartée pour ce sprint, à revoir plus tard

Les « Must » forment le **MVP** (*minimum viable product*) : la plus petite version du produit qui rend déjà service.

8. Tout ça dans GitHub

GitHub a ce qu'il faut pour gérer un petit projet agile :

Notion agile	Dans GitHub
user story, tâche, bug	une **issue**
priorité, type	des **labels** : `must`, `should`, `could`, `bug`…
estimation	un label (`1 pt`, `2 pts`…) ou un champ du Project
sprint	un **milestone** (onglet Issues, puis Milestones)
tableau Kanban	un **Project** au format *Board*

Pour créer le tableau : dans le dépôt, onglet **Projects**, **New project**, choisissez le modèle **Board**. Vous obtenez trois colonnes : *Todo*, *In Progress*, *Done*. Ajoutez une colonne *In Review* pour les PR en attente de relecture. Ajoutez vos issues au tableau, puis déplacez les cartes à la main, ou laissez GitHub le faire : dans le menu ⋯ du Project, **Workflows**, vérifiez que « Item closed » est activé pour qu'une issue fermée passe toute seule dans *Done*.

Exercices du jeudi

J1. Cascade ou agile ? NIVEAU 1

Pour chaque projet, quelle approche choisiriez-vous ? Justifiez en une phrase. Il n'y a pas toujours une seule bonne réponse : c'est la justification qui compte.

1. Le logiciel de calcul des fiches de paie d'une entreprise, qui doit appliquer une nouvelle loi au 1er janvier.
2. Une start-up qui lance une application de partage de plantes d'intérieur entre voisins.
3. Le site vitrine de la boulangerie du quartier : 5 pages, le contenu est déjà prêt.
4. Le jeu vidéo d'un petit studio, dont les joueurs testent une version en accès anticipé.

J2. Qui fait quoi ? NIVEAU 1

Pour chaque situation, dites de quel rôle ou de quel événement Scrum il s'agit.

1. « Hier j'ai fini la commande `ajouter`. Aujourd'hui je fais `supprimer`. Je suis bloqué sur la lecture du fichier. »
2. Elle décide que l'export en PDF passera avant le mode sombre.
3. L'équipe montre la nouvelle version aux utilisateurs, qui demandent de pouvoir trier les tâches par date.
4. Il remarque que les réunions débordent toujours, et propose de les limiter à 15 minutes.
5. L'équipe constate que les PR restent deux jours sans relecture, et décide que chacun relira au moins une PR par matinée.
6. Ils choisissent dans le backlog les six stories qu'ils pensent terminer en deux semaines.

J3. Réparer des user stories NIVEAU 2

Ces stories sont mal écrites. Pour chacune, dites ce qui ne va pas (aidez-vous d'INVEST), puis réécrivez-la.

1. « Faire la base de données. »
2. « En tant qu'utilisateur, je veux que l'application soit bien. »
3. « En tant que joueur, je veux un bouton rouge en haut à droite. »
4. « En tant qu'utilisateur, je veux gérer mes tâches, les partager, les exporter, recevoir des rappels et voir des statistiques. »
5. « En tant que développeur, je veux utiliser des tableaux associatifs. »

J4. Critères d'acceptation NIVEAU 2

Écrivez deux critères d'acceptation *Étant donné / Quand / Alors* pour chacune de ces stories du jeu du pendu :

1. En tant que joueur, je veux proposer une lettre, afin de découvrir le mot petit à petit.
2. En tant que joueur, je veux savoir quand j'ai perdu, afin de connaître le mot et de recommencer.

Pensez au cas normal **et** à un cas limite.

J5. Planning poker NIVEAU 2

En équipe, estimez ces stories d'un « pendu version 2 » avec les valeurs 1, 2, 3, 5, 8, 13. Votez en même temps : chacun écrit son chiffre dans le chat de la visio, et tout le monde appuie sur Entrée au signal.

1. Afficher le pendu en dessin ASCII.
2. Tirer un mot au hasard dans un fichier `mots.txt`.
3. Proposer de rejouer à la fin de la partie.
4. Garder les 10 meilleurs scores dans un fichier, avec le prénom du joueur.
5. Choisir un niveau de difficulté (facile, normal, difficile).
6. Jouer à deux : un joueur choisit le mot, l'autre devine.

Gardez une trace des stories où vos estimations étaient très éloignées au premier vote, et de ce qui vous a mis d'accord.

J6. MoSCoW NIVEAU 2

Vous développez `todo.sh`, un gestionnaire de tâches en ligne de commande, en une journée. Classez ces fonctionnalités en Must, Should, Could et Won't :

ajouter une tâche · lister les tâches · marquer une tâche comme terminée · supprimer une tâche · afficher des statistiques · trier par priorité · synchroniser avec Google Agenda · afficher une aide · ajouter une date limite · une interface web

Puis formulez le MVP en une phrase.

J7. Préparer le sprint de demain NIVEAU 3

C'est l'exercice le plus important de la journée : il prépare le TP de vendredi. En équipe de 3 ou 4 :

1. Lisez les trois sujets du TP (dans le support de vendredi), choisissez-en un, et annoncez-le au formateur.
2. Désignez votre Product Owner et votre Scrum Master. Tout le monde est aussi développeur.
3. Créez le dépôt de l'équipe sur GitHub, invitez tout le monde et le formateur, et protégez `main`.
4. Écrivez **entre 8 et 12 issues** sous forme de user stories, chacune avec au moins un critère d'acceptation.
5. Estimez-les en planning poker, et ajoutez des labels de priorité MoSCoW.
6. Créez le Project *Board* avec les colonnes *Todo*, *In Progress*, *In Review* et *Done*, et ajoutez-y toutes les issues.
7. Écrivez votre Definition of Done dans le README.

Avant 17 h, le formateur passe valider votre backlog. Demain matin, on commence directement par le Sprint Planning.